

TALLER PRÁCTICO

CSS Flexbox & Grid

De cero al layout perfecto — 6 ejercicios progresivos

Nivel: Intermedio | 6 ejercicios | Temas: Flexbox · CSS Grid · Layouts responsivos

¿Para qué sirve este taller?

Al completar los seis ejercicios serás capaz de construir cualquier layout moderno sin recurrir a hacks ni frameworks externos. Cada ejercicio introduce nuevas propiedades de forma incremental, desde la creación básica de un contenedor flex/grid hasta la combinación de ambos en un dashboard real.

CÓMO USAR LOS ARCHIVOS

Dentro de la carpeta `ejercicios/` encontrarás una subcarpeta por ejercicio con `index.html` y `styles.css`. Abre el HTML en el navegador y edita **únicamente** el CSS. Los comentarios `/* TODO */` marcan exactamente qué propiedades debes agregar. Las soluciones están en `soluciones/`.

Parte 1 — CSS Flexbox

¿Qué es Flexbox?

Flexbox (Flexible Box Layout) es un modelo de diseño *unidimensional*: distribuye elementos a lo largo de **un eje** (horizontal o vertical) a la vez. Es ideal para componentes de interfaz: navbars, tarjetas, formularios, barras de herramientas.



Eje principal vs. eje transversal

El **eje principal** (*main axis*) corre en la dirección de `flex-direction` (por defecto `row` = horizontal). El **eje transversal** (*cross axis*) es perpendicular a él. `justify-content` controla el eje principal; `align-items` controla el transversal.

Propiedades del contenedor flex

PROPIEDAD	VALORES CLAVE	EFEECTO
<code>display</code>	<code>flex</code> <code>inline-flex</code>	Activa el contexto flex
<code>flex-direction</code>	<code>row</code> · <code>row-reverse</code> · <code>column</code> · <code>column-reverse</code>	Dirección del eje principal
<code>flex-wrap</code>	<code>nowrap</code> · <code>wrap</code> · <code>wrap-reverse</code>	Permite salto de línea
<code>justify-content</code>	<code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>space-between</code> · <code>space-around</code> · <code>space-evenly</code>	Distribución en eje principal
<code>align-items</code>	<code>stretch</code> · <code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>baseline</code>	Alineación en eje transversal
<code>align-content</code>	Mismos que <code>justify-content</code>	Distribución de filas cuando hay wrap
<code>gap</code>	<code>0.5rem</code> , <code>1rem</code> <code>2rem</code> , ...	Espacio entre ítems

Propiedades de los ítems flex

PROPIEDAD	VALORES CLAVE	EFEECTO
<code>flex-grow</code>	<code>0</code> (no crece) · <code>1</code> (crece)	Factor de crecimiento
<code>flex-shrink</code>	<code>1</code> (encoge) · <code>0</code> (no encoge)	Factor de encogimiento
<code>flex-basis</code>	<code>auto</code> · <code>200px</code> · <code>30%</code>	Tamaño base antes de distribuir espacio
<code>flex</code>	<code>flex: grow shrink basis</code>	Shorthand de los tres anteriores
<code>align-self</code>	Mismos que <code>align-items</code>	Anula <code>align-items</code> para un solo ítem
<code>order</code>	Entero (por defecto <code>0</code>)	Orden visual (sin cambiar el HTML)

 TRUCO — SHORTHAND FLEX

`flex: 1` equivale a `flex: 1 1 0` : el ítem crece, encoge y su base es 0 (ocupa espacio proporcional). `flex: none` equivale a `flex: 0 0 auto` : tamaño fijo, no crece ni encoge.

Parte 2 — CSS Grid

¿Qué es CSS Grid?

CSS Grid es un modelo *bidimensional*: controla simultáneamente **filas y columnas**. Es perfecto para el layout macro de una página: headers, sidebars, grillas de contenido.



Conceptos clave

- **Línea de cuadrícula (grid line):** las líneas numeradas que dividen el grid (empiezan en 1).
- **Track:** el espacio entre dos líneas — una fila o una columna.
- **Celda:** intersección de un track de fila y uno de columna.
- **Área:** rectángulo de una o más celdas contiguas.
- **fr (fracción):** unidad que representa una fracción del espacio disponible.

Propiedades del contenedor grid

PROPIEDAD	EJEMPLO	EFECTO
display	grid	Activa el contexto grid
grid-template-columns	repeat(3, 1fr)	Define las columnas
grid-template-rows	60px 1fr auto	Define las filas
grid-template-areas	"header header" "sidebar main"	Nombra áreas rectangulares
grid-auto-rows	160px	Altura de filas implícitas
grid-auto-columns	1fr	Ancho de columnas implícitas
gap	1rem	Espacio entre celdas

Propiedades de los ítems grid

PROPIEDAD	EJEMPLO	EFECTO
grid-area	header	Asigna el ítem al área nombrada
grid-column	1 / 3 o span 2	Columna inicial / final
grid-row	2 / 4 o span 2	Fila inicial / final
align-self	start · end · center · stretch	Alineación vertical del ítem

<code>justify-self</code>	<code>start · end · center · stretch</code>	Alineación horizontal del ítem
---------------------------	---	--------------------------------

Funciones útiles

FUNCIÓN	EJEMPLO	DESCRIPCIÓN
<code>repeat()</code>	<code>repeat(4, 1fr)</code>	Repite un track N veces
<code>minmax()</code>	<code>minmax(200px, 1fr)</code>	Track con tamaño mínimo y máximo
<code>auto-fill</code>	<code>repeat(auto-fill, minmax(200px,1fr))</code>	Crea tantas columnas como quepan
<code>auto-fit</code>	igual que auto-fill	Como auto-fill pero colapsa columnas vacías

```
/* Layout clásico de página con áreas */
.layout {
  display: grid;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  grid-template-columns: 250px 1fr;
  grid-template-rows: 64px 1fr auto;
  min-height: 100vh;
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

Flexbox vs. Grid — ¿cuándo usar cada uno?

USA FLEXBOX CUANDO...

- El layout es **unidimensional** (fila o columna)
- Necesitas distribuir ítems en una **barra de navegación**
- Quieres centrar contenido vertical y horizontalmente
- Los ítems tienen **tamaños variables** y quieres que crezcan/encojan
- Construyes un componente: card, formulario, lista de tags

USA GRID CUANDO...

- El layout es **bidimensional** (filas y columnas)
- Construyes el **esqueleto de una página**: header, sidebar, footer
- Necesitas que ítems **salten celdas** o abarquen varias
- Quieres columnas que se creen automáticamente con **auto-fill**
- Diseñas una **galería** o un tablero tipo dashboard

✓ REGLA DE ORO

No son excluyentes: **Grid para el layout macro** (estructura de la página) y **Flexbox para los componentes** (interior de cada bloque). En el Ejercicio 6 los usarás juntos.

Los 6 ejercicios del taller

01

Barra de Navegación

Básico

Flexbox `display · align-items · justify-content · gap`

Objetivo: construir una navbar horizontal donde el logo queda a la izquierda, los links al centro y el botón "Contacto" al extremo derecho.

PROPIEDADES QUE PRACTICAS

- `display: flex` — activa el contenedor flex
- `align-items: center` — centra verticalmente
- `justify-content: space-between` — empuja el botón al extremo
- `gap` — espacio entre links

TAREA BONUS

Prueba `margin-left: auto` en el botón como alternativa a `space-between`. ¿Cuál prefieres? ¿Por qué?

ARCHIVOS

[ejercicios/01-flex-navegacion/styles.css](#)

02

Tarjetas de Productos

Básico-Medio

Flexbox `flex-wrap · flex-basis · flex-grow · dirección columna`

Objetivo: crear una grilla de tarjetas que se ajuste automáticamente: máximo 3 por fila, precio siempre al fondo de cada tarjeta.

PROPIEDADES QUE PRACTICAS

- `flex-wrap: wrap` — permite salto de línea
- `flex-basis: calc(33.333% - 1rem)` — tamaño base para 3 columnas
- `flex-grow: 1` — cada tarjeta llena el espacio sobrante
- `flex-direction: column` — dentro de la tarjeta para empujar el precio

CONCEPTO CLAVE — FLEX EN COLUMNA

La tarjeta usa `display: flex; flex-direction: column`. Al poner `flex-grow: 1` en la descripción, ella se expande y empuja el precio al fondo, sin importar cuánto texto tenga cada tarjeta.

ARCHIVOS

[ejercicios/02-flex-tarjetas/styles.css](#)

03

Perfil de Usuario — Flexbox Avanzado

Medio

Flexbox flex shrink/grow · align-self · order

Objetivo: practicar las propiedades de los ítems individuales en tres escenarios reales.

TRES PARTES

1. **Tarjeta de perfil:** el avatar no se encoge (`flex-shrink: 0`), la info crece (`flex: 1`), el botón se alinea arriba (`align-self: flex-start`).
2. **Reordenar con `order`** : el botón "Seguir" debe aparecer visualmente antes del nombre, sin mover el HTML.
3. **Barra de búsqueda:** el ícono es fijo (`flex: 0 0 auto`), el input crece (`flex-grow: 1`).

ARCHIVOS

[ejercicios/03-flex-avanzado/styles.css](#)

04

Galería de Fotos

Básico Grid

Grid repeat · fr · auto-fill · minmax · grid-column/row

Objetivo: aprender las bases de CSS Grid creando una galería en tres variantes.

TRES PARTES

1. **Grid fijo:** 4 columnas iguales con `repeat(4, 1fr)` , filas automáticas de 160 px.
2. **Ítems destacados:** una foto ocupa 2 columnas (`grid-column: span 2`), otra ocupa 2 filas (`grid-row: span 2`).
3. **Responsivo:** columnas automáticas con `repeat(auto-fill, minmax(200px, 1fr))` . Cambia el ancho del navegador para ver el efecto.

ARCHIVOS

[ejercicios/04-grid-basico/styles.css](#)

05

Layout de Página Completo

Medio Grid

Grid grid-template-areas · grid-area · responsive

Objetivo: construir el layout clásico header / sidebar / main / footer usando `grid-template-areas` .

PROPIEDADES QUE PRACTICAS

- `grid-template-areas` — nombra zonas con cadenas de texto
- `grid-template-columns: 250px 1fr` — sidebar fijo, main flexible
- `grid-template-rows: 64px 1fr auto` — header fijo, contenido flexible, footer adaptable
- `grid-area` en cada hijo

TAREA BONUS (RESPONSIVO)

Con un `@media (max-width: 768px)` redefine las áreas para móvil: el sidebar pasa debajo del main y se usa una sola columna.

ARCHIVOS

[ejercicios/05-grid-areas/styles.css](#)

06 Dashboard Completo — CSS desde cero

Flexbox

Grid

Integración completa · CSS libre

Avanzado

Objetivo: escribir *todo* el CSS desde cero para un dashboard con topbar, sidebar, KPIs, gráficas y tabla de actividad. El archivo `styles.css` está intencionalmente vacío.

QUÉ DEBES IMPLEMENTAR

- ☐ Layout general con Grid y `grid-template-areas`
- ☐ Topbar con Flexbox y `justify-content: space-between`
- ☐ Sidebar con Flexbox en columna: nav arriba, usuario abajo
- ☐ KPIs con `repeat(auto-fill, minmax(180px, 1fr))`
- ☐ Charts con Grid 2 columnas (2fr / 1fr)
- ☐ Barra de barras simuladas con Flexbox y `align-items: flex-end`
- ☐ Tabla de actividad con header flex

ARCHIVOS

[ejercicios/06-flex-grid-dashboard/styles.css](#) (vacío — a rellenar)



SIN MIRAR LAS SOLUCIONES PRIMERO

Intenta completar el dashboard por tu cuenta. Solo consulta [soluciones/06-flex-grid-dashboard/styles.css](#) si llevas más de 20 minutos atascado en algo.

Referencia rápida — hoja de trucos

Flexbox en una imagen

```
/* Contenedor */
.container {
  display: flex;
  flex-direction: row | column;
  flex-wrap: wrap;
  justify-content: flex-start | center | space-between | space-around | space-evenly;
  align-items: stretch | flex-start | flex-end | center | baseline;
  gap: 1rem;
}

/* Ítems */
.item {
  flex: 1 1 0;          /* grow shrink basis */
  flex-grow: 1;          /* ocupa espacio sobrante */
  flex-shrink: 0;        /* no se encoge */
  flex-basis: 200px;     /* tamaño base */
  align-self: flex-end;  /* alineación individual */
  order: -1;             /* aparece primero visualmente */
}
```

CSS Grid en una imagen

```
/* Contenedor */
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: 60px 1fr auto;
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
  grid-auto-rows: minmax(100px, auto);
  gap: 1rem;
}

/* Ítems */
.item {
  grid-area: header;      /* asigna área nombrada */
  grid-column: 1 / 3;     /* de línea 1 a 3 */
  grid-column: span 2;    /* abarca 2 columnas */
  grid-row: span 2;       /* abarca 2 filas */
  justify-self: center;   /* alinea horizontalmente */
  align-self: end;        /* alinea verticalmente */
}
```

```
/* Grid responsivo */
.responsive-grid {
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
}
```

Patrones comunes

PATRÓN	CSS
Centrar contenido (Flex)	display:flex; justify-content:center; align-items:center;
Empujar un ítem al extremo	margin-left: auto en el ítem que quieres a la derecha
Tarjetas de igual altura	display:flex en el contenedor (stretch por defecto)
Precio al fondo de la tarjeta	flex-direction:column + flex-grow:1 en la descripción
Galería responsiva	repeat(auto-fill, minmax(200px, 1fr))
Layout de página	grid-template-areas + grid-area
Ítem que abarca 2 cols	grid-column: span 2
Sidebar responsivo	@media + redefinir grid-template-areas

Criterios de evaluación

EJERCICIO	CRITERIO	PTS.
01 — Navbar	Navbar muestra logo · links · botón correctamente distribuidos	10
02 — Tarjetas	3 cols, wrap correcto, precio al fondo en todas las tarjetas	15
03 — Avanzado	Partes A/B/C funcionan: shrink, order, grow en barra	15
04 — Galería	3 partes: grid fijo, span, responsive con auto-fill	15
05 — Layout	Áreas correctas; BONUS: responsive en móvil	20
06 — Dashboard	CSS completo desde cero, estructura visual coherente	25
Total		100



NOTA PARA EL DOCENTE

Los ejercicios 01-03 pueden hacerse en la primera sesión (Flexbox). Los ejercicios 04-05 en la segunda sesión (Grid). El ejercicio 06 es el proyecto integrador y puede asignarse como tarea o hacerse en pares en clase. Las soluciones están en la carpeta **soluciones/** y pueden mostrarse al final de cada sesión.